Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №2**

По дисциплине «Аппаратное и программное обеспечение сетей»

**Тема: «Организация UDP – сервера»**

**Выполнил:**
Студент 3 курса
Группы ИИ-27
Титунин А.А,
**Проверил:**
Кулик А.Д.

Брест 2026

Вариант 4

Цель: изучить принцип работы UDP и научится работать с сокетами.

Задание:

10.1 Изучить теоретический материал, функции и классы пакета java.net и листинг программы реализации UDP-сервера и UDP-клиента. Получить индивидуальное задание у преподавателя.
10.2 Разработать программу работы UDP-сервера, выполняющую функции согласно варианта задания к лабораторной работе 1 (см. приложения).
10.3 Разработать программу работы UDP-клиента (см. приложения).
10.4 Выполнить проверку программ UDP – сервера и UDP – клиента, используя адрес 127.0.0.1 и порт 666. Продемонстрировать работу системы преподавателю.
10.5 Представить отчет, содержащий титульный лист, листинг программы с подробными комментариями основных фрагментов программ.10.6 Подготовиться к защите лабораторной работы по теоретическому материалу, функциям, собственным результатам, полученным в ходе выполнения лабораторной работы.

Задание варианта:
После приема каждой группы из 64 символов от клиента сервер формирует и отсылает клиенту цепочку <«символ1-количество1», «символ2-количество2» и т.д.>, где «символ» - встреченный в группе символ, а «количество» - их количество в группе. Если в очередной группе число разных символов меньше 3, то отсылается соответствующее сообщение клиенту и сервер разрывает соединение.

Клиент должен отсылать введенные символы по нажатию клавиши Home, а также устанавливать соединение при помощи команды connect и разрывать его при помощи команды disconnect. Клиент должен вести лог отдельным текстовым файлом, куда записываются время установления соединения, время разрыва соединения и информация, переданная от сервера.

Ход работы:
Пользуясь методическими указаниями я написал программу сервер и программу клиент согласно своему варианту на языке java в среде разработки Visual Studio Code с использованием Java.net.
Сервер запускается и открывает соединение, после чего к нему подключается клиент при помощи команды connect, и отправляет введенные пользователем символы, которые собираются в блоки по 64 байта на стороне сервера. Далее сервер возвращает количество каждого символа в блоке, или разрывает соединение, если разных символов 3 или меньше.

Листинг программы-сервера:

```
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.*;
import java.util.ArrayList;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
```

```
class Pair {
    int number;
    char letter;

    Pair(char letter, int number) {
        this.letter = letter;
        this.number = number;
    }
}

public class UdpServer {
    public static final int PORT = 8080;
    public static final int BUFFER_SIZE = 64;

    private JFrame frame;
    private JTextArea log;
    private JLabel status;
    private JButton startButton;
    private JButton stopButton;

    private DatagramSocket socket;
    private volatile boolean running = false;
    private Thread serverThread;

    // Состояние на каждого клиента — аналог отдельного потока в TCP-версии
    private static class ClientState {
        StringBuilder buffer = new StringBuilder();
        ArrayList<Pair> pairs = new ArrayList<>();
    }

    private final Map<String, ClientState> clients = new ConcurrentHashMap<>();

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new UdpServer().buildUI());
    }

    private void buildUI() {
        frame = new JFrame("UDP Server");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        log = new JTextArea(22, 55);
        log.setEditable(false);
        log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));

        status = new JLabel("Status: stopped");
        startButton = new JButton("Start");
        stopButton = new JButton("Stop");
        stopButton.setEnabled(false);

        startButton.addActionListener(e -> startServer());
        stopButton.addActionListener(e -> stopServer());

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.LEFT));
        buttons.add(startButton);
        buttons.add(stopButton);
        buttons.add(status);

        frame.getContentPane().setLayout(new BorderLayout(5, 5));
        frame.getContentPane().add(new JScrollPane(log), BorderLayout.CENTER);
        frame.getContentPane().add(buttons, BorderLayout.SOUTH);

        frame.pack();
```

```
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        appendLog("[SERVER] Ready. Press Start to listen on UDP port " + PORT + ".");
    }

    private void startServer() {
        if (running) return;

        try {
            socket = new DatagramSocket(PORT);
        } catch (IOException e) {
            appendLog("[SERVER] Cannot open port " + PORT + ": " + e.getMessage());
            return;
        }

        running = true;
        clients.clear();
        startButton.setEnabled(false);
        stopButton.setEnabled(true);
        status.setText("Status: listening on port " + PORT);
        appendLog("[SERVER] Started: " + socket.getLocalSocketAddress());

        serverThread = new Thread(this::receiveLoop);
        serverThread.setDaemon(true);
        serverThread.start();
    }

    private void receiveLoop() {
        byte[] buf = new byte[4096];
        while (running) {
            try {
                DatagramPacket packet = new DatagramPacket(buf, buf.length);
                socket.receive(packet);
                handlePacket(packet);
            } catch (IOException e) {
                if (running) {
                    appendLog("[SERVER] Receive error: " + e.getMessage());
                }
                if (socket.isClosed()) break;
            }
        }
    }

    private void handlePacket(DatagramPacket packet) {
        InetAddress addr = packet.getAddress();
        int port = packet.getPort();
        String key = addr.getHostAddress() + ":" + port;

        String msg;
        try {
            msg = new String(packet.getData(), packet.getOffset(),
                    packet.getLength(), "UTF-8");
        } catch (UnsupportedEncodingException e) {
            return;
        }

        // маркер завершения сессии от клиента (аналог EOF в TCP)
        if (msg.equals("__disconnect__")) {
            appendLog("[SERVER] Client " + port + " disconnected.");
            clients.remove(key);
            return;
```

```
    }

    ClientState state = clients.computeIfAbsent(key, k -> {
        appendLog("[SERVER] Connection accepted: " + k);
        return new ClientState();
    });

    // Накапливаем символы в буфер (как in.read() в TCP-версии)
    for (int i = 0; i < msg.length(); i++) {
        state.buffer.append(msg.charAt(i));
        if (state.buffer.length() == BUFFER_SIZE) {
            processBlock(state, addr, port);
            if (!clients.containsKey(key)) {
                // сервер закрыл клиента внутри processBlock
                return;
            }
        }
    }
}

private void processBlock(ClientState state, InetAddress addr, int port) {
    char[] buffer = state.buffer.toString().toCharArray();

    for (char c : buffer) {
        boolean isExist = false;
        for (int i = 0; i < state.pairs.size(); i++) {
            if (state.pairs.get(i).letter == c) {
                state.pairs.get(i).number++;
                isExist = true;
                break;
            }
        }
        if (!isExist) {
            state.pairs.add(new Pair(c, 1));
        }
    }

    appendLog("[SERVER] Block from " + port + ": " + new String(buffer));

    if (state.pairs.size() <= 3) {
        appendLog("[SERVER] -> not enough characters, closing client " + port);
        reply(addr, port, "not enough characters");
        reply(addr, port, "closing...");
        clients.remove(addr.getHostAddress() + ":" + port);
        return;
    } else {
        appendLog("[SERVER] -> answer:");
        for (int i = 0; i < state.pairs.size(); i++) {
            String line = "Character " + state.pairs.get(i).letter
                    + ": " + state.pairs.get(i).number;
            appendLog("[SERVER]    " + line);
            reply(addr, port, line);
        }
    }

    // сброс буфера и счётчиков (как в TCP-версии)
    state.buffer.setLength(0);
    state.pairs.clear();
}

private void reply(InetAddress addr, int port, String text) {
    try {
```

```
            byte[] out = text.getBytes("UTF-8");
            DatagramPacket p = new DatagramPacket(out, out.length, addr, port);
            socket.send(p);
        } catch (IOException e) {
            appendLog("[SERVER] Send error: " + e.getMessage());
        }
    }

    private void stopServer() {
        if (!running) return;
        running = false;

        try {
            if (socket != null && !socket.isClosed()) {
                socket.close();
            }
        } catch (Exception ignored) {}

        clients.clear();
        startButton.setEnabled(true);
        stopButton.setEnabled(false);
        status.setText("Status: stopped");
        appendLog("[SERVER] Stopped.");
    }

    private void appendLog(String line) {
        SwingUtilities.invokeLater(() -> {
            log.append(line + "\n");
            log.setCaretPosition(log.getDocument().getLength());
        });
    }
}
}
```

Листинг программы-клиента:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class UdpClient {
    private DatagramSocket socket;
    private InetAddress serverAddress;
    private int serverPort;

    private JTextField inputField;
    private JTextArea log;
    private JLabel status;

    private static final String LOG_FILE = "client_log.txt";
    private static final DateTimeFormatter TIME_FMT =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private PrintWriter fileLog;

    private static final int BUFFER_SIZE = 4096;

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new UdpClient().buildUI());
    }
```

```
private void buildUI() {
    try {
        fileLog = new PrintWriter(new BufferedWriter(
                new FileWriter(LOG_FILE, true)), true);
    } catch (IOException e) {
        System.err.println("Cannot open log file: " + e.getMessage());
    }

    JFrame frame = new JFrame("UDP Client");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    inputField = new JTextField(40);
    log = new JTextArea(20, 55);
    log.setEditable(false);
    log.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));
    status = new JLabel("Status: disconnected");

    JLabel hint = new JLabel(
            "Commands: connect <host> <port> | disconnect | exit    "
            + "— other input is sent to server. Press HOME to execute/send.");

    inputField.addKeyListener(new KeyAdapter() {
        @Override
        public void keyPressed(KeyEvent e) {
            if (e.getKeyCode() == KeyEvent.VK_HOME) {
                handleInput();
                e.consume();
            }
        }
    });

    inputField.addActionListener(e -> handleInput());

    JPanel top = new JPanel(new BorderLayout(5, 5));
    top.add(inputField, BorderLayout.NORTH);
    top.add(hint, BorderLayout.SOUTH);

    frame.getContentPane().setLayout(new BorderLayout(5, 5));
    frame.getContentPane().add(top, BorderLayout.NORTH);
    frame.getContentPane().add(new JScrollPane(log), BorderLayout.CENTER);
    frame.getContentPane().add(status, BorderLayout.SOUTH);

    frame.addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            closeEverything();
        }
    });

    frame.pack();
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);

    appendLog("[CLIENT] Ready. Use 'connect <host> <port>' to connect.");
    inputField.requestFocusInWindow();
}

private void handleInput() {
    String line = inputField.getText().trim();
    inputField.setText("");
    if (line.isEmpty()) return;
```

```
    String[] parts = line.split("\\s+");
    String first = parts[0].toLowerCase();

    if (first.equals("connect")) {
        handleConnect(parts);
    } else if (first.equals("disconnect")) {
        handleDisconnect();
    } else if (first.equals("exit")) {
        handleDisconnect();
        appendLog("[CLIENT] Bye.");
        closeEverything();
        System.exit(0);
    } else {
        sendData(line);
    }

    inputField.requestFocusInWindow();
}

private void handleConnect(String[] parts) {
    if (socket != null && !socket.isClosed()) {
        appendLog("[CLIENT] Already connected to "
                + serverAddress + ":" + serverPort
                + ". Use 'disconnect' first.");
        return;
    }

    if (parts.length != 3) {
        appendLog("[CLIENT] Usage: connect <host> <port>");
        return;
    }

    String host = parts[1];
    int port;
    try {
        port = Integer.parseInt(parts[2]);
    } catch (NumberFormatException e) {
        appendLog("[CLIENT] Invalid port: " + parts[2]);
        return;
    }

    try {
        serverAddress = InetAddress.getByName(host);
        serverPort = port;
        socket = new DatagramSocket();

        appendLog("[CLIENT] Connected to " + serverAddress + ":" + serverPort);
        status.setText("Status: connected to " + serverAddress + ":" + serverPort);

        writeFileLog("CONNECTION START: " + LocalDateTime.now().format(TIME_FMT));

        startReaderThread();

    } catch (IOException e) {
        appendLog("[CLIENT] Cannot connect: " + e.getMessage());
        socket = null;
        serverAddress = null;
    }
}

private void handleDisconnect() {
```

```
    if (socket == null || socket.isClosed()) {
        appendLog("[CLIENT] Not connected.");
        return;
    }
    try {
        writeFileLog("CONNECTION END:   " + LocalDateTime.now().format(TIME_FMT));

        // в TCP при close() сервер узнаёт о разрыве. В UDP шлём маркер.
        sendRaw("__disconnect__");

        socket.close();
        appendLog("[CLIENT] Disconnected.");
    } catch (IOException e) {
        appendLog("[CLIENT] Error on disconnect: " + e.getMessage());
    } finally {
        socket = null;
        serverAddress = null;
        status.setText("Status: disconnected");
    }
}

private void sendData(String text) {
    if (socket == null || socket.isClosed() || serverAddress == null) {
        appendLog("[CLIENT] Not connected. Use 'connect <host> <port>' first.");
        return;
    }
    sendRaw(text);
    appendLog("[CLIENT] Sent " + text.length() + " characters: " + text);
}

private void sendRaw(String text) {
    try {
        byte[] buf = text.getBytes("UTF-8");
        DatagramPacket packet = new DatagramPacket(
                buf, buf.length, serverAddress, serverPort);
        socket.send(packet);
    } catch (IOException e) {
        appendLog("[CLIENT] Send error: " + e.getMessage());
    }
}

private void startReaderThread() {
    final DatagramSocket currentSocket = socket;

    Thread t = new Thread(() -> {
        byte[] buf = new byte[BUFFER_SIZE];
        try {
            while (!currentSocket.isClosed()) {
                DatagramPacket packet = new DatagramPacket(buf, buf.length);
                currentSocket.receive(packet);

                if (!packet.getAddress().equals(serverAddress)
                        || packet.getPort() != serverPort) {
                    continue;
                }

                String response = new String(
                        packet.getData(), packet.getOffset(),
                        packet.getLength(), "UTF-8");

                final String r = response;
                SwingUtilities.invokeLater(() -> appendLog("[SERVER] " + r));
```

```
                writeFileLog("SERVER: " + r);

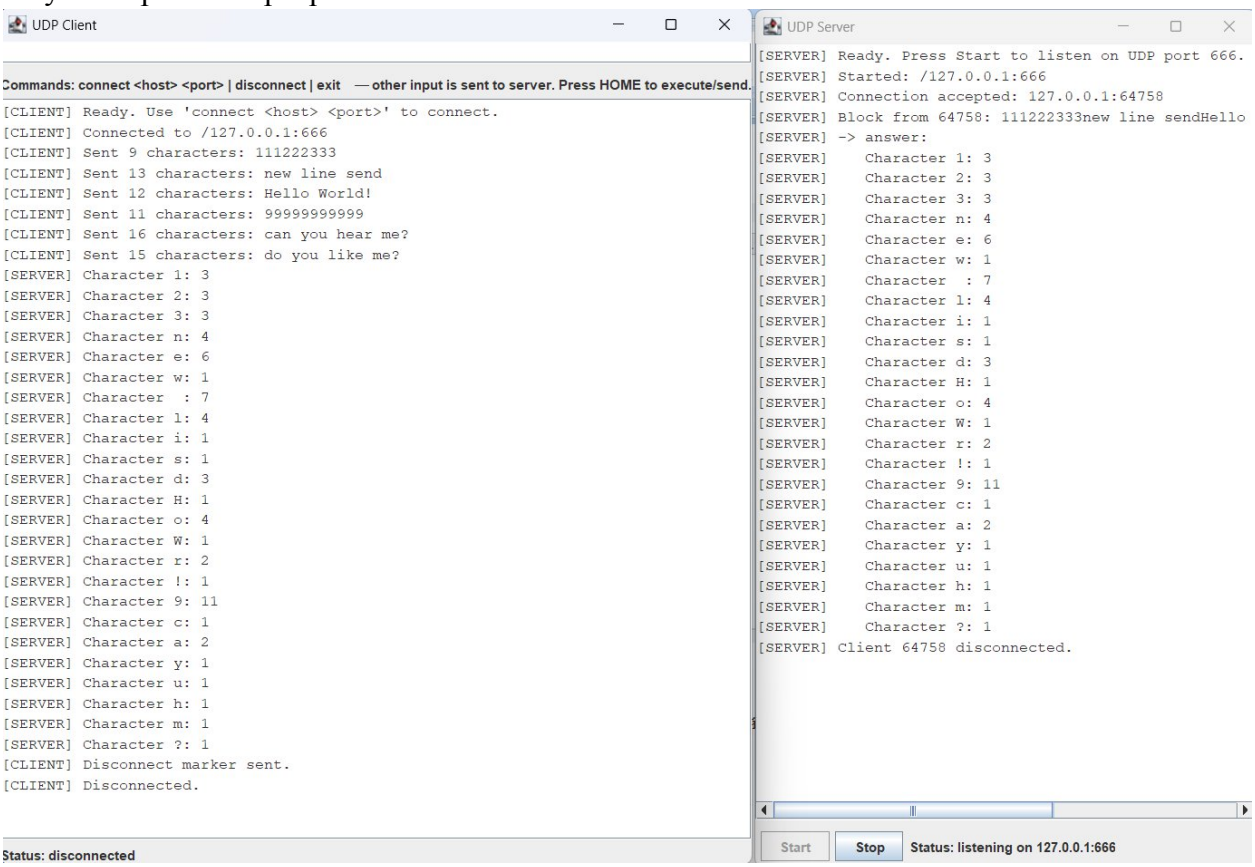
                if (response.equals("closing...")) {
                    SwingUtilities.invokeLater(() -> {
                        appendLog("[CLIENT] Server closed the connection.");
                        status.setText("Status: disconnected");
                    });
                    writeFileLog("CONNECTION END:   "
                            + LocalDateTime.now().format(TIME_FMT));
                    try { currentSocket.close(); } catch (Exception ignored) {}
                    socket = null;
                    serverAddress = null;
                    return;
                }
            }
        } catch (IOException e) {
            if (currentSocket.isClosed() || socket == null) {
                // закрытие по нашей инициативе
            } else {
                SwingUtilities.invokeLater(() ->
                        appendLog("[CLIENT] Read error: " + e.getMessage()));
            }
        }
    });
    t.setDaemon(true);
    t.start();
  }

  private void appendLog(String line) {
    SwingUtilities.invokeLater(() -> {
        log.append(line + "\n");
        log.setCaretPosition(log.getDocument().getLength());
    });
  }

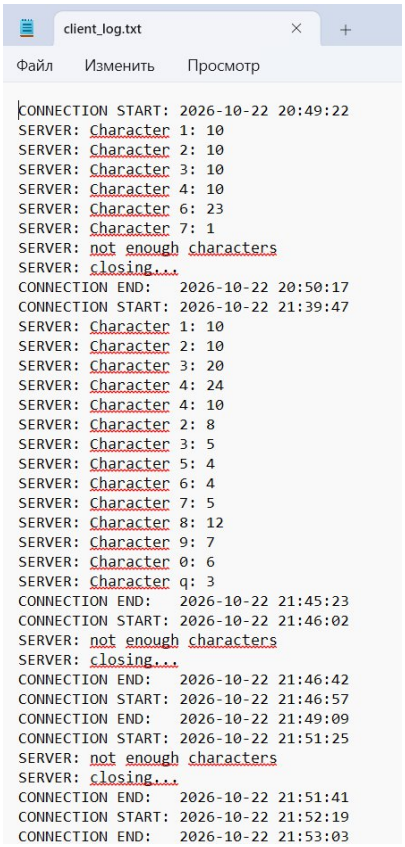
  private synchronized void writeFileLog(String line) {
    if (fileLog == null) return;
    fileLog.println(line);
  }

  private void closeEverything() {
    if (socket != null && !socket.isClosed()) {
        writeFileLog("CONNECTION END:   " + LocalDateTime.now().format(TIME_FMT));
        try { socket.close(); } catch (Exception ignored) {}
    }
    if (fileLog != null) {
        fileLog.close();
        fileLog = null;
    }
  }
}
```

Результат работы программ:



Лог:




Вывод: в ходе выполнения лабораторной работы я познакомился с UDP и научился работать с сокетами.